

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN
FACULTAD DE INGENIERÍA — ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS
Curso: Diseño de Sistemas — Ciclo 03

ESTÁNDARES DE ARQUITECTURA Y DESARROLLO

Sistema Web de Gestión de Inventario y Ventas
“SYSELL” — Comercial Machi's

Arquitectura: Clean Architecture + CQRS | Sistema web

Código de documento	STD-ARQ-SYSELL-WEB-2026
Versión	2.0 — Adecuación a sistema web
Proyecto de referencia	SYSELL — Sistema de ventas e inventario para Comercial Machi's
Equipo	Pollitos Design
Fecha de emisión	28 de septiembre de 2026

CUADRO DE REVISIONES

Este documento es el estándar de arquitectura y desarrollo del sistema web SYSELL. Conserva Clean Architecture + CQRS como patrón central y define un sistema web accesible desde cualquier dispositivo con navegador, apoyado en una API REST y una base de datos centralizada.

N° Revisión	Fecha	Motivo del cambio	Elaborado por	Aprobado
1.0	22/09/2026	Versión inicial del estándar de arquitectura y desarrollo.	Equipo Pollitos Design	Pendiente
2.0	28/09/2026	Adecuación al sistema web: frontend web, API REST, seguridad web y diseño responsive.	Equipo Pollitos Design	Pendiente

ÍNDICE

1. OBJETO

Establecer el estándar de arquitectura y desarrollo del sistema SYSELL como sistema web, definiendo los lineamientos, patrones y buenas prácticas de arquitectura, codificación, base de datos, seguridad e interfaz que garanticen uniformidad, mantenibilidad, seguridad y calidad. El documento fusiona los estándares institucionales entregados por el docente (Anexo 9-29 MCVS-604, Manual SYAR, Estándares de Desarrollo de BD y Guías UNJBG 2026), sustituyendo el patrón MVC/BFF por Clean Architecture + CQRS, y los adecúa al proyecto SYSELL de Comercial Machi's.

2. FINALIDAD

Uniformizar el desarrollo de SYSELL como aplicación web accesible desde cualquier dispositivo con navegador, manteniendo la lógica de negocio centralizada en el backend. Cualquier integrante del equipo debe poder comprender y extender el código sin ambigüedad sobre la organización de capas, la nomenclatura o el flujo de datos.

3. ÁMBITO DE APLICACIÓN

El estándar es de aplicación obligatoria para todos los componentes del sistema web SYSELL desarrollados por Pollitos Design: el cliente web responsive (Administrador y Vendedor), la API REST y la base de datos MySQL.

Permanecen fuera del alcance: pasarela de pago en línea, integración con SUNAT y comprobantes electrónicos, módulo de compras a proveedores y tienda virtual para clientes externos.

4. MARCO METODOLÓGICO Y REFERENCIAS

4.1. Clean Architecture

Organiza el software en capas concéntricas (Dominio, Aplicación, Infraestructura y Presentación) con la Regla de Dependencia: las capas internas nunca dependen de las externas. Así, las reglas de negocio de SYSELL (control de stock, ganancias, roles) no dependen del navegador, del framework ni de la base de datos.

4.2. CQRS

Separa las operaciones que modifican el estado (Commands, por ejemplo registrar una venta) de las que solo consultan (Queries, por ejemplo consultar stock o un reporte). En el frontend web se refleja como consultas y mutaciones independientes.

4.3. API-first

El backend se diseña como una API REST independiente de la interfaz. El cliente web consume los endpoints de la API y toda la regla de negocio reside en el backend.

4.4. ISO/IEC 25010 y DevSecOps

La calidad se evalúa con los ocho atributos de ISO/IEC 25010 definidos en el Informe de Producto Final, y la seguridad se incorpora desde el diseño (Shift Left), atendiendo las observaciones de las auditorías internas y externas del equipo.

5. ARQUITECTURA GENERAL DEL SISTEMA

5.1. Arquitectura cliente-servidor API-first

SYSELL adopta una arquitectura cliente-servidor: un backend único que expone la API REST y una base de datos centralizada, consumidos por el cliente web. Los dos locales de Comercial Machi's trabajan sobre la misma información en tiempo real (R-09, R-15).

Componente	Responsabilidad	Tecnología propuesta
Cliente web	Interfaz responsive para Administrador y Vendedor, accesible desde PC, tablet o celular.	React + TypeScript + Vite
API REST (backend)	Casos de uso como Commands y Queries; reglas de negocio, seguridad y validación.	PHP 8.2+ con Laravel
Base de datos	Persistencia centralizada de usuarios, productos, ventas y reportes.	MySQL 8
Servicios de soporte	Respaldo en la nube y exportación de reportes (PDF, Excel, CSV).	Almacenamiento en nube; librerías de exportación en el backend

5.2. Requisitos de plataforma (requerimientos no funcionales)

Aspecto	Requisito
Plataforma	Navegadores modernos: Chrome, Edge, Firefox y Safari (últimas 2 versiones)
Recursos del dispositivo	Cualquier dispositivo con navegador y conexión a internet
Diseño de pantalla	Responsive desde 360 px de ancho hasta pantallas de PC
Seguridad	Roles, respaldo en nube y protección web (HTTPS, CSRF, XSS, CORS)
Escalabilidad	Más usuarios, productos y locales; la API permite crecer sin cambios de negocio

5.3. Módulos del sistema

Módulo	Requerimientos	Casos de uso	Actor principal
Usuario	R-01 a R-05	CU.1 Validar usuario, registrar, ver, editar y eliminar perfil	Administrador
Inventario	R-10 a R-14	CU.3 Registrar, modificar, mostrar y dar de baja producto	Administrador
Salida de producto	R-06 a R-09	CU.2 Realizar venta, seleccionar método de pago, consultar disponibilidad	Vendedor / Administrador
Reporte de salida	R-15 a R-18	CU.4 Total por local, tablas y gráficos, filtrar y exportar	Administrador

6. CLEAN ARCHITECTURE: CAPAS Y RESPONSABILIDADES

6.1. Backend (API REST)

Capa	Contenido	Puede depender de	Ejemplos SYSELL
Dominio	Entidades, objetos de valor, reglas de negocio puras, interfaces de repositorio.	Nada	Usuario, Producto, Venta, Stock; regla “no vender sin stock”
Aplicación	Commands, Queries, Handlers, DTOs.	Dominio	RegistrarSalidaProductoCommand, ObtenerStockQuery
Infraestructura	Repositorios Eloquent/MySQL, hashing, exportadores.	Aplicación, Dominio	EloquentProductoRepository, ExportadorPdfReporte
Presentación	Controllers REST, Form Requests, Resources, middleware.	Aplicación	VentaController

- El Dominio no conoce HTTP, Laravel ni MySQL; las entidades protegen sus invariantes (stock no negativo, DNI de 8 dígitos).
- Los controllers son delgados: validan la entrada, crean el Command o Query y lo envían al bus.
- Los Handlers reciben y devuelven DTOs, nunca modelos de persistencia.

6.2. Frontend web

El cliente web también se organiza por capas para no acoplar la lógica de negocio a React ni a la API:

Capa	Contenido	Ejemplo
Domain	Tipos y modelos TypeScript, reglas simples de cliente.	Producto.ts, MetodoPago.ts
Application	Casos de uso: hooks de consulta y mutación.	useStockProducto, useRegistrarVenta
Infrastructure	Cliente HTTP, mapeo de DTOs, almacenamiento de sesión.	ventaApi.ts, httpClient.ts
Presentation	Páginas y componentes de interfaz, sin lógica de negocio.	VentaPage.tsx, TablaProductos.tsx

- Regla: el frontend nunca es la única barrera de seguridad; toda validación de rol y de negocio se repite en el backend.

7. PATRÓN CQRS

7.1. Principios

- Un Command modifica el estado, se nombra en imperativo y devuelve a lo sumo un identificador.
- Una Query solo lee y devuelve DTOs de lectura optimizados; nunca modifica datos.
- Cada Command o Query tiene exactamente un Handler.
- En el frontend, las Queries se implementan como consultas con caché (GET) y los Commands como mutaciones (POST, PUT, DELETE) que invalidan la caché afectada.

7.2. Commands y Queries de SYSELL

Módulo	Commands (escritura)	Queries (lectura)
Usuario	IniciarSesionCommand, RegistrarUsuarioCommand, EditarUsuarioCommand, EliminarUsuarioCommand	ObtenerPerfilUsuarioQuery, ListarUsuariosQuery
Inventario	RegistrarProductoCommand, ModificarProductoCommand, AgregarStockCommand, DarBajaProductoCommand	ObtenerStockProductoQuery, BuscarProductosQuery, ObtenerDetalleProductoQuery
Salida de producto	RegistrarSalidaProductoCommand (descuenta stock, registra método de pago y ganancia), GestionarDevolucionCommand	ConsultarDisponibilidadQuery, ListarVentasRecientesQuery
Reporte	ExportarReporteCommand	ObtenerTotalVentasPorLocalQuery , ObtenerEstadisticasSalidasQuery, FiltrarReporteVentasQuery

7.3. Flujo de un Command web: Realizar venta (CU.2)

1. En VentaPage el vendedor confirma la venta; el hook useRegistrarVenta envía POST /api/v1/ventas.
2. VentaController valida la entrada y crea RegistrarSalidaProductoCommand.
3. El CommandBus lo entrega a RegistrarSalidaProductoHandler.
4. El Handler verifica stock (regla de Dominio), crea la Venta, descuenta stock y registra método de pago y ganancia en una sola transacción.
5. Se responde 201 Created; el frontend invalida la caché de stock y ventas recientes.

7.4. Flujo de una Query web: Consultar disponibilidad (R-09)

1. El vendedor escribe en el buscador; useStockProducto envía GET /api/v1/productos/{id}/stock.
2. El controller crea ConsultarDisponibilidadQuery y la envía al QueryBus.
3. El QueryHandler lee de la BD y devuelve un DTO (nombre, talla, color, stock, precio) con 200 OK.

8. ESTRUCTURA DE CARPETAS

8.1. Repositorio único (monorepo)

```
sysell/
├── backend/      (API Laravel: Clean Architecture + CQRS)
├── web/          (cliente web React + TypeScript)
├── docs/         (estándares, diagramas, contrato OpenAPI)
└── docker-compose.yml
```

8.2. Backend

```
backend/app/
├── Domain/       (Usuario/, Inventario/, Venta/, Shared/)
├── Application/  (Usuario/, Inventario/, Venta/, Reporte/ con Commands/, Queries/,
Handlers/, DTOs/; Bus/)
├── Infrastructure/ (Persistence/, Security/, Export/)
└── Presentation/ (Http/Controllers, Requests, Resources, Middleware)
```

8.3. Cliente web

```
web/src/
├── app/          (rutas, proveedores, configuración global)
├── features/     (auth/, usuarios/, inventario/, ventas/, reportes/)
│   └── cada feature: domain/, application/, infrastructure/, presentation/
├── shared/       (componentes UI, hooks, utilidades, tema)
└── assets/
```

9. ESTÁNDAR DE NOMENCLATURA Y CODIFICACIÓN

9.1. Backend PHP

Elemento	Convención	Ejemplo
Estándar base	PSR-12 y autoloader PSR-4	namespace App\Application\Venta\Commands;
Clases e interfaces	PascalCase; interfaces con sufijo Interface	ProductoRepositoryInterface
Commands	Verbo + Entidad + Command	RegistrarProductoCommand
Queries	Obtener/Listar/Buscar + Entidad + Query	ListarUsuariosQuery
Handlers	Nombre del Command/Query + Handler	RegistrarProductoHandler
Métodos y variables	camelCase	\$stockDisponible, calcularTotal()
Constantes	MAYUSCULAS_CON_GUION_BAJO	STOCK_MINIMO
Controllers y Requests	Entidad + Controller; Acción + Entidad + Request	VentaController, RegistrarUsuarioRequest

9.2. Frontend web (TypeScript / React)

Elemento	Convención	Ejemplo
Componentes React	PascalCase, un componente por archivo	TablaProductos.tsx
Hooks	prefijo use + camelCase	useRegistrarVenta
Funciones y variables	camelCase	stockDisponible
Tipos e interfaces	PascalCase, sin prefijo I	Producto, VentaDto
Carpetas y archivos no componente	kebab-case o camelCase de forma consistente	venta-api.ts
Constantes	MAYUSCULAS_CON_GUION_BAJO	API_BASE_URL
Estilos	Tailwind CSS o CSS Modules; sin estilos en línea salvo casos dinámicos	className="btn-primary"
Herramientas	ESLint + Prettier obligatorios; TypeScript en modo estricto	npm run lint

9.3. Reglas generales

- El dominio de negocio se nombra en español (Producto, Venta) por trazabilidad con los requerimientos; los términos técnicos en inglés (Repository, Handler).
- Una clase o componente, una responsabilidad; funciones de máximo 30 líneas como referencia.
- Prohibido código muerto, código comentado y credenciales en el código fuente.

- URLs, claves y secretos van en variables de entorno (.env), nunca versionadas.

10. ESTÁNDAR DE BASE DE DATOS (MySQL)

10.1. Nomenclatura general

Objeto	Regla	Ejemplo
Tablas	snake_case, singular, minúsculas	usuario, producto, detalle_venta
Columnas	snake_case descriptivo	fecha_registro, precio_venta
Clave primaria	id (BIGINT UNSIGNED AUTO_INCREMENT)	producto.id
Clave foránea	entidad_id	venta.usuario_id
Índices y restricciones	idx_, uq_, fk_ + tabla y columna	idx_producto_nombre, uq_usuario_dni
Booleanos	prefijo es_ o tiene_	es_activo
Auditoría	created_at, updated_at, deleted_at (borrado lógico)	producto.deleted_at

10.2. Modelo de tablas de SYSELL

Tabla	Campos principales	Requerimiento
rol	id, nombre (Administrador, Vendedor)	R-02
usuario	id, rol_id, local_id, nombres, apellidos, dni (único), correo, telefono, direccion, password_hash, es_activo	R-01 a R-05
local	id, nombre, direccion	R-15
categoria	id, nombre	R-12
producto	id, categoria_id, nombre, marca, material, talla, color, precio_venta, precio_costo, imagen_url, es_activo	R-10, R-12 a R-14
stock	id, producto_id, local_id, cantidad	R-06, R-09, R-11
metodo_pago	id, nombre (efectivo, billetera virtual, transferencia)	R-08
venta	id, usuario_id, local_id, metodo_pago_id, total, ganancia, fecha_venta	R-06 a R-08
detalle_venta	id, venta_id, producto_id, cantidad, precio_unitario	R-06, R-07

10.3. Reglas de diseño

- Mínimo tercera forma normal; integridad referencial obligatoria con claves foráneas.
- Borrado lógico de usuarios y productos para conservar el historial de ventas (riesgo detectado en R-04 y R-14).
- El stock se actualiza solo desde casos de uso de Aplicación, dentro de una transacción.
- Contraseñas únicamente como hash; cambios de esquema solo mediante migraciones versionadas; motor InnoDB y utf8mb4.

11. ESTÁNDAR DE API REST

11.1. Principios

- La API es el contrato único entre el cliente web y el backend; toda regla de negocio se resuelve en el servidor.
- El contrato se documenta con OpenAPI (Swagger) en docs/; todo cambio de endpoint actualiza el contrato.
- Versionado en la URL (/api/v1); los cambios que rompen compatibilidad crean /api/v2.
- Listados con paginación (page, per_page) y filtros por query string.

11.2. Endpoints

GET corresponde a Queries; POST, PUT/PATCH y DELETE a Commands.

Método	Endpoint	Tipo CQRS	Rol permitido
POST	/api/v1/auth/login	Command	Público
POST	/api/v1/usuarios	Command	Administrador
GET	/api/v1/usuarios/{id}	Query	Administrador, Vendedor (propio)
PUT	/api/v1/usuarios/{id}	Command	Administrador
DELETE	/api/v1/usuarios/{id}	Command	Administrador
POST	/api/v1/productos	Command	Administrador
GET	/api/v1/productos?buscar=	Query	Administrador, Vendedor
PUT	/api/v1/productos/{id}	Command	Administrador
DELETE	/api/v1/productos/{id}	Command	Administrador
POST	/api/v1/ventas	Command	Administrador, Vendedor
GET	/api/v1/reportes/ventas	Query	Administrador
GET	/api/v1/reportes/ventas/exportar?formato=pdf	Query	Administrador

11.3. Formato de respuesta y códigos HTTP

```
{ "success": true, "message": "Venta registrada correctamente",
  "data": { "id": 128 }, "errors": null }
```

Código	Uso
200 / 201 / 204	Consulta o modificación correcta / recurso creado / sin contenido
400 / 422	Solicitud mal formada / error de validación (DNI inválido, stock insuficiente)
401 / 403	No autenticado / sin permiso para el rol
404 / 409	Recurso no encontrado / conflicto (empleado ya registrado)

Código	Uso
429 / 500	Demasiadas solicitudes / error interno sin exponer detalles técnicos

11.4. CORS

La API solo acepta solicitudes del dominio del cliente web autorizado (lista blanca por entorno) y exige autenticación en todos los endpoints salvo el login.

12. ESTÁNDAR DE SEGURIDAD WEB

Responde a las observaciones del Informe de Auditoría N.º 1 (expiración de sesiones, cifrado de contraseñas y validación de permisos por rol) y adopta como referencia OWASP Top 10.

Control	Lineamiento
Autenticación	Login obligatorio (R-02). Web: sesión con Laravel Sanctum en modo SPA con cookie HttpOnly, Secure y SameSite. Expiración por inactividad de 8 horas y revocación al cerrar sesión.
Autorización por rol	Roles Administrador y Vendedor validados con middleware y policies en cada endpoint. Ocultar botones en la interfaz no sustituye la validación del servidor.
Contraseñas	Hash con bcrypt o Argon2, mínimo 8 caracteres; nunca en respuestas ni logs; límite de intentos de login.
Transporte	HTTPS/TLS obligatorio y encabezado HSTS.
CSRF y XSS	Protección CSRF de Sanctum; escape de salida por defecto de React; prohibido dangerouslySetInnerHTML sin sanitizar; encabezados de seguridad (Content-Security-Policy, X-Content-Type-Options).
Inyección SQL	Solo consultas parametrizadas (Eloquent / Query Builder).
Validación	Form Requests en backend; validación previa en el frontend solo como ayuda de usabilidad.
Datos sensibles	DNI, teléfono y dirección visibles solo para el Administrador o el propio usuario.
Limitación de tasa	Rate limiting en login y exportaciones.
Respaldo y secretos	Copia diaria en la nube con prueba de restauración; secretos en variables de entorno.
Trazabilidad	Registro de auditoría de altas, bajas y modificaciones de usuarios, productos y ventas.

13. ESTÁNDAR DE INTERFAZ WEB Y USABILIDAD

Aspecto	Lineamiento
Enfoque responsive	Mobile-first: se diseña primero para 360 px y se amplía. Puntos de quiebre: 640, 768, 1024 y 1280 px.
Diseño	Prototipos en Figma como fuente de verdad; colores, tipografías y espaciados definidos como tokens de diseño.
Componentes	Biblioteca de componentes reutilizables en shared/ (botones, tablas, formularios, modales de confirmación).
Accesibilidad	Contraste mínimo WCAG 2.1 AA, navegación por teclado, etiquetas en formularios.
Acciones destructivas	Confirmación obligatoria antes de eliminar perfiles o productos (problema identificado en el análisis de R-04 y R-14).
Rendimiento	Carga inicial (LCP) menor a 2.5 s; paginación de tablas; carga diferida de gráficos y reportes.
Reportes	Gráficos con una librería estándar (por ejemplo Recharts o Chart.js); filtros visibles y exportación a PDF, Excel y CSV.
Navegadores soportados	Últimas 2 versiones de Chrome, Edge, Firefox y Safari.
Idioma	Interfaz en español; textos centralizados para facilitar cambios.

14. CONTROL DE VERSIONES, CI/CD Y DESPLIEGUE

- **Ramas:** main (producción), develop (integración), feature/<modulo>-<descripcion>, fix/<descripcion>, release/<version>.
- **Commits:** Conventional Commits en español, ej.: feat(venta): registrar salida de producto con descuento de stock.
- **Pull Requests:** obligatorios hacia develop, con al menos una revisión cruzada de otro integrante.
- **Integración continua:** cada PR ejecuta compilación, lint, pruebas y análisis estático (SonarQube o Codacy).
- **Entornos:** local (Docker Compose), staging (réplica de producción con BD de prueba) y producción.
- **Despliegue:** desde main tras pasar CI; el frontend se compila a archivos estáticos y el backend se despliega con migraciones automáticas versionadas.
- **Prohibido:** push directo a main o develop y subir archivos .env, claves o binarios.
- **Roles del equipo:** Líder (Johan Mamani), Diseñador (Luis Condori), Programador (Aron Cachicatari), Testeador (Alex Chata), Analista (Pierrette Chávez).

15. ESTÁNDAR DE PRUEBAS Y CALIDAD

Tipo de prueba	Alcance	Herramienta sugerida	Meta
Unitarias backend	Entidades y Handlers (stock, ganancias, roles)	PHPUnit	≥ 80 % en Dominio y Aplicación
Unitarias frontend	Hooks, utilidades y componentes	Vitest + Testing Library	≥ 70 % en lógica
Integración	Repositorios y endpoints contra BD de pruebas	PHPUnit (Feature)	Todos los endpoints
Extremo a extremo	Flujos CU.1 a CU.4 en staging y en varios navegadores	Playwright	Flujos críticos cubiertos
Regresión	Reejecución tras cada cambio	CI (GitHub Actions)	100 % en verde para fusionar
Usabilidad	Registro de venta rápido para el personal	Sesiones con usuarios reales	Venta en menos de 1 minuto

Se dispone de un entorno de staging con base de datos propia, para evitar la limitación reportada en la auditoría N.º 3 (no poder probar todas las pantallas por falta de servicio de base de datos).

15.1. Atributos ISO/IEC 25010 aplicados

Atributo	Cómo lo garantiza este estándar
Adecuación funcional	Cada requerimiento R-01 a R-18 se traza a un Command o Query.
Eficiencia de desempeño	Consultas de lectura optimizadas con CQRS, paginación y carga diferida en la web.
Compatibilidad	API REST en JSON consumible por cualquier navegador moderno.
Usabilidad	Diseño responsive, confirmaciones y mensajes claros (sección 13).
Fiabilidad	Transacciones en ventas y respaldo diario en la nube.
Seguridad	Sección 12: roles, hash, HTTPS, CSRF y auditoría.
Mantenibilidad	Clean Architecture, nomenclatura uniforme, revisión de código y análisis estático.
Portabilidad	Web accesible desde cualquier dispositivo; backend independiente del cliente.

16. GLOSARIO

Término	Definición
Clean Architecture	Arquitectura en capas concéntricas donde el negocio no depende de frameworks ni de la base de datos.
CQRS	Separación de las operaciones de escritura (Commands) y lectura (Queries).

Término	Definición
API-first	Diseñar primero la API como contrato para que cualquier cliente la consuma.
SPA	Aplicación de una sola página que se ejecuta en el navegador.
Responsive	Diseño que se adapta a distintos tamaños de pantalla.
CORS / CSRF / XSS	Mecanismos y ataques web: control de orígenes, falsificación de solicitudes y scripts maliciosos.
Handler / DTO	Clase que ejecuta un Command o Query / objeto simple para transportar datos entre capas.
Borrado lógico	Marcar un registro como eliminado (deleted_at) sin borrarlo físicamente.
Staging	Entorno de pruebas semejante a producción.

17. REFERENCIAS

- Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
- Cohaila Gonzales, M. G. (2026). Diapositivas y material del curso Diseño de Sistemas. ESIS, UNJBG.
- Pressman, R. S. (2010). Ingeniería del software: un enfoque práctico (7.ª ed.). McGraw-Hill.
- ISO/IEC 25010: Systems and software engineering, Software product quality model.
- OWASP Foundation. OWASP Top 10 (proyecto de seguridad de aplicaciones web).
- Documentos base del docente: Anexo 9-29 MCVS-604, Manual de Estándares SYAR, Estándares de Desarrollo (BD) y Guías UNJBG 2026 v2 y v3.
- Equipo Pollitos Design (2026). Informe Nro. 1 y Trabajo Final de Unidad I, proyecto SYSELL.